

## PROJECT DESIGN — OVERVIEW

### 1. Design Objective

The design defines how the web interface, REST APIs, authentication, FAQ management, AI matching engine and MongoDB data layer work together.

### 2. Design Principles

| Simplicity      | Clear interface for students and administrators.                |
|-----------------|-----------------------------------------------------------------|
| Modularity      | Separate routes, controllers, models, middleware and utilities. |
| Security        | JWT authentication and admin authorization.                     |
| Maintainability | AI matching logic is isolated in a utility.                     |
| Scalability     | REST APIs and structured MongoDB data.                          |
| Usability       | Relevant answers and suggestions are presented clearly.         |

### 3. Project Team

| Team ID     | SWTID-2026-9997 |
|-------------|-----------------|
| Team Size   | 5               |
| Team Leader | Shalini K       |
| Team Member | Sindhuja S      |
| Team Member | Sabrin Nisha K  |
| Team Member | Yamini A        |
| Team Member | Sankari N       |



## TECHNICAL ARCHITECTURE

### 1. Architecture Overview

Users interact through browser pages. Frontend JavaScript communicates with Express REST APIs. Controllers and middleware process requests, the AI utility performs FAQ matching, and Mongoose communicates with MongoDB.



### 2. Component Responsibilities

| Presentation | HTML pages and client-side JavaScript.      |
|--------------|---------------------------------------------|
| API          | Express REST routes.                        |
| Business     | Controllers implement operations.           |
| Security     | JWT authentication and admin authorization. |
| AI           | aiEngine.js ranks FAQ matches.              |
| Persistence  | Mongoose models access MongoDB.             |

DATA & DATABASE DESIGN

1. Entity Relationship Design

The main entities are User, Category and FAQ. FAQs are associated with categories, while user roles control access.



2. Entity Details

| User     | name, email, password, role, createdAt                  | Authentication and access control. |
|----------|---------------------------------------------------------|------------------------------------|
| Category | name, description, createdAt                            | Organizes FAQs.                    |
| FAQ      | question, answer, category, keywords, views, timestamps | Stores searchable FAQ content.     |

## DATA FLOW DESIGN

### 1. System Data Flow

The request begins at the browser, passes through the relevant Express route and controller, accesses the AI engine or database, and returns a structured response.



### 2. Major Request Flows

| Authentication | Browser → Auth Route → Controller → User Model → MongoDB → JWT / Response  |
|----------------|----------------------------------------------------------------------------|
| FAQ Retrieval  | Browser → FAQ Route → Controller → FAQ Model → MongoDB → Response          |
| Admin Update   | Admin → Protected Route → Middleware → Controller → MongoDB → Response     |
| AI Question    | User → AI Route → AI Controller → AI Engine → FAQ Data → Result → Response |

## AI MATCHING DESIGN

### 1. AI Engine Overview

The project uses a custom JavaScript similarity engine in `src/utils/aiEngine.js`. It is not a generative AI API; it compares user text with stored FAQ information and ranks candidate matches.



### 2. Matching Stages

| Tokenization        | Converts text into comparable tokens.                   |
|---------------------|---------------------------------------------------------|
| Stop-word Filtering | Removes common low-value words.                         |
| Exact / Near Match  | Checks similarity with FAQ questions.                   |
| Keyword Matching    | Compares terms against FAQ keywords.                    |
| Token Matching      | Compares question and answer tokens.                    |
| Coverage Bonus      | Rewards broader term coverage.                          |
| Ranking             | Sorts candidates by score.                              |
| Threshold           | Returns relevant result only when threshold is reached. |
| Suggestions         | Provides related FAQ suggestions when available.        |

## API & MODULE DESIGN

### 1. API Modules

| Authentication | POST /api/auth/register; POST /api/auth/login; GET /api/auth/me | Register/login public; /me protected |
|----------------|-----------------------------------------------------------------|--------------------------------------|
| FAQs           | GET/POST /api/faqs; GET/PUT/DELETE /api/faqs/:id                | GET public; mutations admin          |
| Categories     | GET/POST /api/categories; GET/PUT/DELETE /api/categories/:id    | GET public; mutations admin          |
| Users          | GET /api/users; stats; DELETE; role update                      | Admin                                |
| AI             | POST /api/ai/ask                                                | AI endpoint                          |

### 2. Module Structure

| Routes      | src/routes/                    | REST endpoints                   |
|-------------|--------------------------------|----------------------------------|
| Controllers | src/controllers/               | Business logic                   |
| Models      | src/models/                    | MongoDB schemas                  |
| Middleware  | src/middleware/                | Auth + errors                    |
| Utility     | src/utls/aiEngine.js           | AI matching                      |
| Frontend    | public/*.html + public/js/*.js | User interfaces and interactions |

SECURITY & ROLE DESIGN

1. Security Design

| Password Protection | bcryptjs password hashing.                 |
|---------------------|--------------------------------------------|
| Authentication      | JWT tokens.                                |
| Authorization       | Protected routes and admin checks.         |
| Sensitive Data      | Password excluded from normal JSON output. |
| API Protection      | FAQ/category/user mutations protected.     |
| Error Handling      | Centralized error middleware.              |

2. Role-Based Access Matrix



| Browse FAQs        | ✓ | ✓ |
|--------------------|---|---|
| View Categories    | ✓ | ✓ |
| Ask AI             | ✓ | ✓ |
| Manage FAQs        | — | ✓ |
| Manage Categories  | — | ✓ |
| Manage Users/Roles | — | ✓ |

## ERROR HANDLING & SCALABILITY

### 1. Error Handling

| Invalid login     | Authentication error without sensitive information. |
|-------------------|-----------------------------------------------------|
| Missing token     | Reject protected request.                           |
| Insufficient role | Reject administrative operation.                    |
| Invalid resource  | Return API error.                                   |
| No AI match       | Return no-match message and suggestions.            |
| Unexpected error  | Centralized error middleware.                       |

### 2. Scalability

| Database | MongoDB supports additional records.            |
|----------|-------------------------------------------------|
| Backend  | Modular routes/controllers allow new modules.   |
| AI       | Matching logic can be enhanced independently.   |
| Frontend | Separate pages/scripts support future features. |
| API      | REST structure allows future clients.           |
| Search   | Can later extend to semantic/NLP methods.       |

### 3. Future Enhancements

Multilingual support, voice input, improved semantic search, conversation history, feedback-based learning and advanced analytics can be incorporated in future versions.

## COMPLETE DESIGN FLOW

### 1. End-to-End Flow

| Student / Admin | HTML + JS | REST Routes | JWT + Business Logic | Matching / CRUD | Users / Categories / FAQs |
|-----------------|-----------|-------------|----------------------|-----------------|---------------------------|

### 2. Design Traceability

| Secure access       | JWT + bcryptjs + authentication middleware |
|---------------------|--------------------------------------------|
| FAQ organization    | Category model + routes                    |
| FAQ management      | FAQ model + controller/routes + admin UI   |
| Intelligent answers | AI controller + aiEngine.js                |
| Related suggestions | AI matching results                        |
| Role control        | User role + admin authorization            |
| Persistent data     | MongoDB + Mongoose                         |

### 3. Design Conclusion

The design separates presentation, API, authentication, data management, AI matching and error handling into identifiable components, providing a maintainable foundation for the AI FAQ Assistant.